

Object Oriented Programming – CS/CE 224/272

Nadia Nasir



Habib University, Fall 2024

Content

- Variable storage in Memory
 - Cast and type conversion
 - Functions – Revisited
 - Arrays
 - Passing arrays in a function
 - Enum
-

Variable Storage in Memory (C++)

Memory Basics

- Bit = Smallest unit of data: 0 or 1
 - Byte = 8 bits (e.g., 01001101)
 - Memory is structured in bytes.
 - Each byte has an address (like a house number).
-
- When you declare a variable in C++, memory is allocated to store its value.

```
int a = 10; //declaring an int a
```

- int typically takes 4 bytes (32 bits).
 - These 4 bytes are reserved in RAM.
 - The variable a is stored at a specific memory address (e.g., 0x1000).
-

Integer 'a' occupies 4 bytes starting from address 0x1000.
In little-endian systems, the least significant byte is stored first.

Memory Address	Content (Byte)

0x1000	00000000
0x1001	00000000
0x1002	00000000
0x1003	00001010 ← 10 in binary

Casting

- We have seen type conversion in python.
- Casts are used to obtain a different type e.g.
 - If you want to convert int type to double type
 - Or float type to double type
- There are quite a few casts available in C++.
- For now we will only use 'static_cast<typename>(var)'

```
int main()
{
    int i = 0;
    double d = static_cast<double>(i);

    double dd = 0.1;
    int ii = static_cast<int>(dd);
    char c = static_cast<double>(dd);

    bool b = true;
    double ddd = static_cast<double>(b);
}
```

Comparing double and float

- Floating point equal comparisons are unreliable!!

```
int main()
{
    double a = (5. * 0.3) + 0.1;
    double b = 1.6;

    if(a == b)
    {
        std::cout << "a == b" << '\n';
    }
    else
    {
        std::cout << "Magic" << '\n';
    }

    return 0;
}
```

```
int main()
{
    double a = (3. * 0.3) + 0.1;
    double b = 1;

    if(a == b)
    {
        std::cout << "a == b" << '\n';
    }
    else
    {
        std::cout << "Magic" << '\n';
    }

    return 0;
}
```

Comparing double and float

- You should compare floating point to a certain precision

```
int main()
{
    double b = 3 * 0.3 + 0.1;
    double a = 1.;

    double epsilon = 1.e-6;
    if(std::abs(a - b) < epsilon)
    {
        std::cout << "a == b" << '\n';
    }
    else
    {
        std::cout << "Magic" << '\n';
    }

    return 0;
}
```

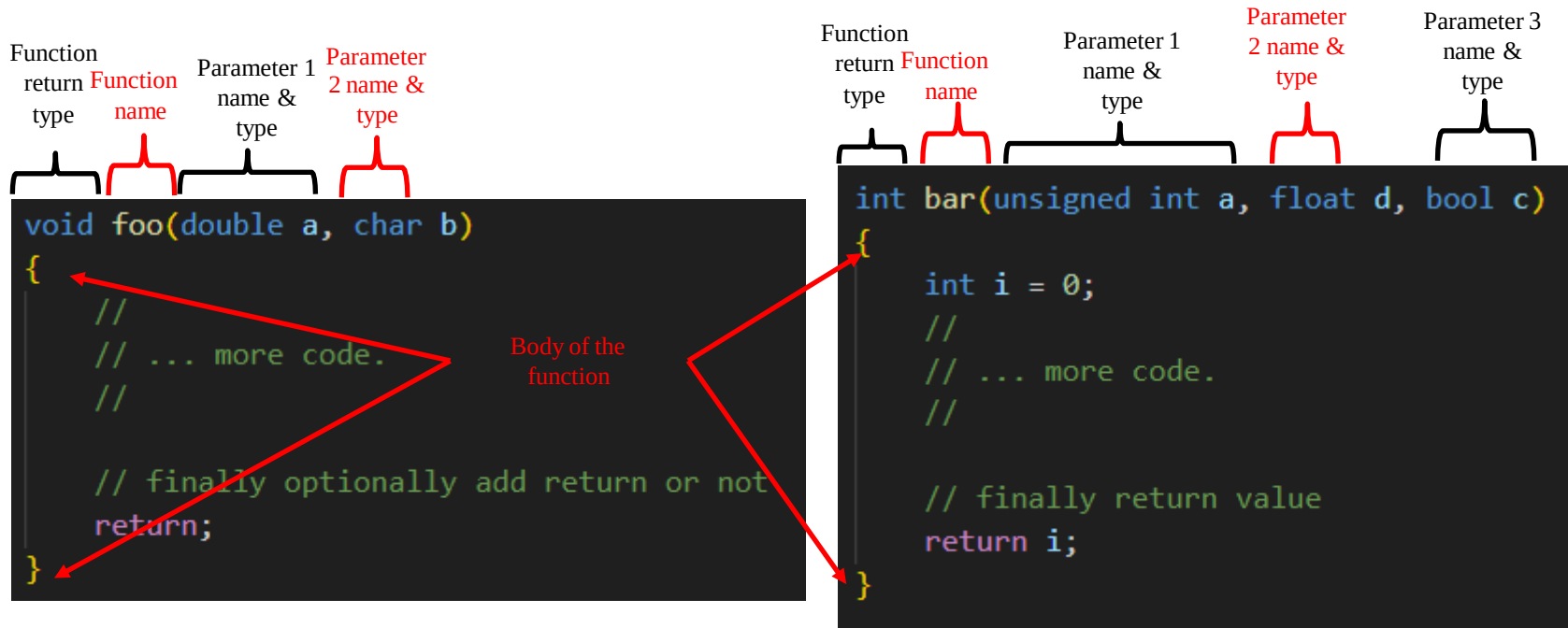
- You can read up on other ways to do floating point comparison

Functions revisited

We will come back to functions in this course many times!

Functions in C++ = functions in python

- Functions help organise your code logically
- Prevents duplication of code
- C++ function syntax



Declaration vs definition

- A function without body is called a declaration

Note semicolon

```
double computeArea(double a, double b);  
  
int main()  
{  
    double area = computeArea(2., 5.);  
    return 0;  
}
```

- Where is the function body?
-

Declaration vs definition

- Declaration and definition in one shot
 - **computeArea is declared prior to call from main.** This is what you have seen before

```
// function declared and defined in one shot
double computeArea(double a, double b)
{
    //
    // ... more code.
    //

    // finally optionally add return or not
    return 0.;
}

int main()
{
    // compute area at least declared prior to the call
    double area = computeArea(2., 5.);
    return 0;
}
```

Declaration vs definition

- In LHS code, function declared and defined in one shot
 - Error: **FUNCTION MUST BE DECLARED before it is called**
- In RHS code, function declared first and defined after main
 - No Error: because computeArea **DECLARED BEFORE the call to main**

```
int main()
{
    double area = computeArea(2., 5.);
    return 0;
}

double computeArea(double a, double b)
{
    //                                     CODE DOESN'T
    // ... more code.                     COMPILER
    //

    // finally optionally add return or not
    return 0.;
}
```

```
double computeArea(double a, double b);

int main()
{
    double area = computeArea(2., 5.);
    return 0;
}

double computeArea(double a, double b)
{
    //                                     CODE COMPILES
    // ... more code.
    //

    // finally optionally add return or not
    return 0.;
}
```

Declaration vs definition

- We usually “DECLARE” functions in the header file
 - and the function is “DEFINED” in the source file
 - Separating declaration and definition
 - reduces code pollution hence improves readability
 - Improves compile time
 - Enables you to hide implementation details
 - We will see separating declaration and definitions in different files in future! But remember this concept
-

Arrays in C++

We will use arrays in this course many times!

Arrays aka python list (sort of)

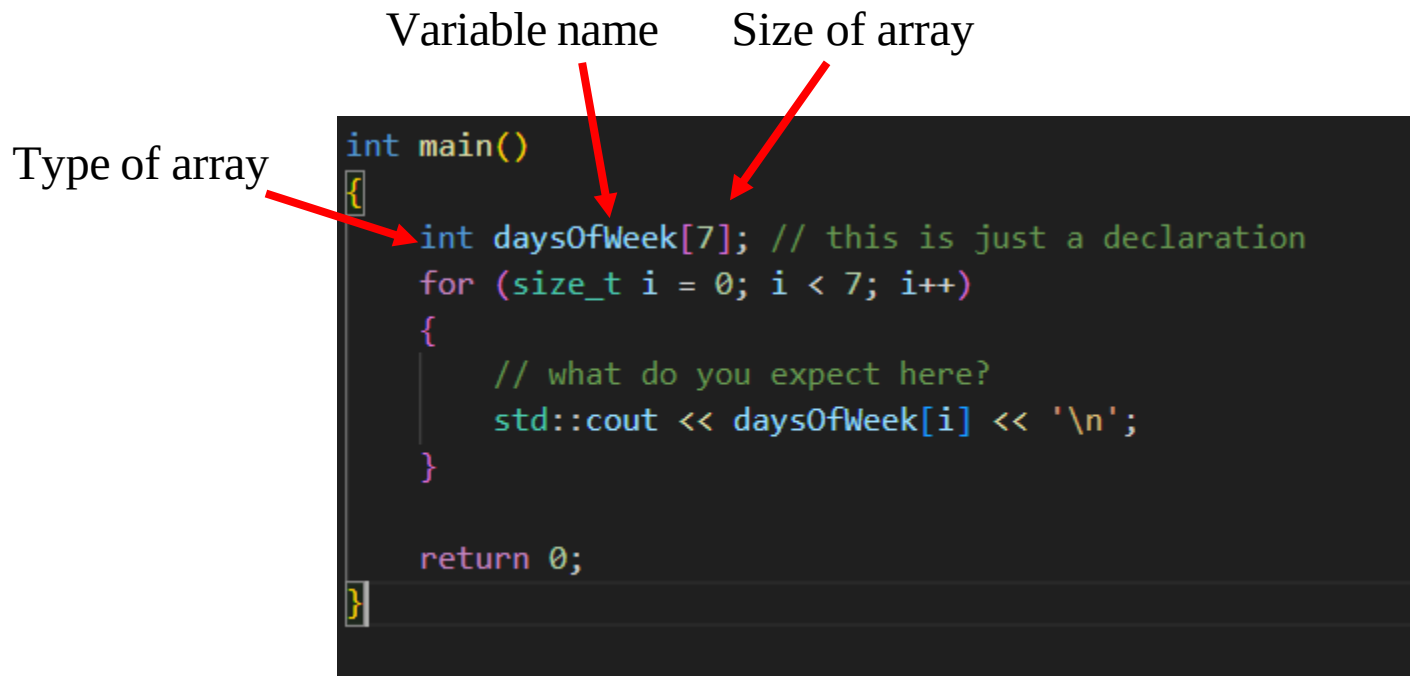
- Arrays are homogenous data structure.
- You need to know the size of the array upfront

Variable name Size of array

Type of array

```
int main()
{
    int daysOfWeek[7]; // this is just a declaration
    for (size_t i = 0; i < 7; i++)
    {
        // what do you expect here?
        std::cout << daysOfWeek[i] << '\n';
    }

    return 0;
}
```



- What output do you see?

Arrays

- Arrays are homogenous data structure.
- You need to know the size of the array upfront

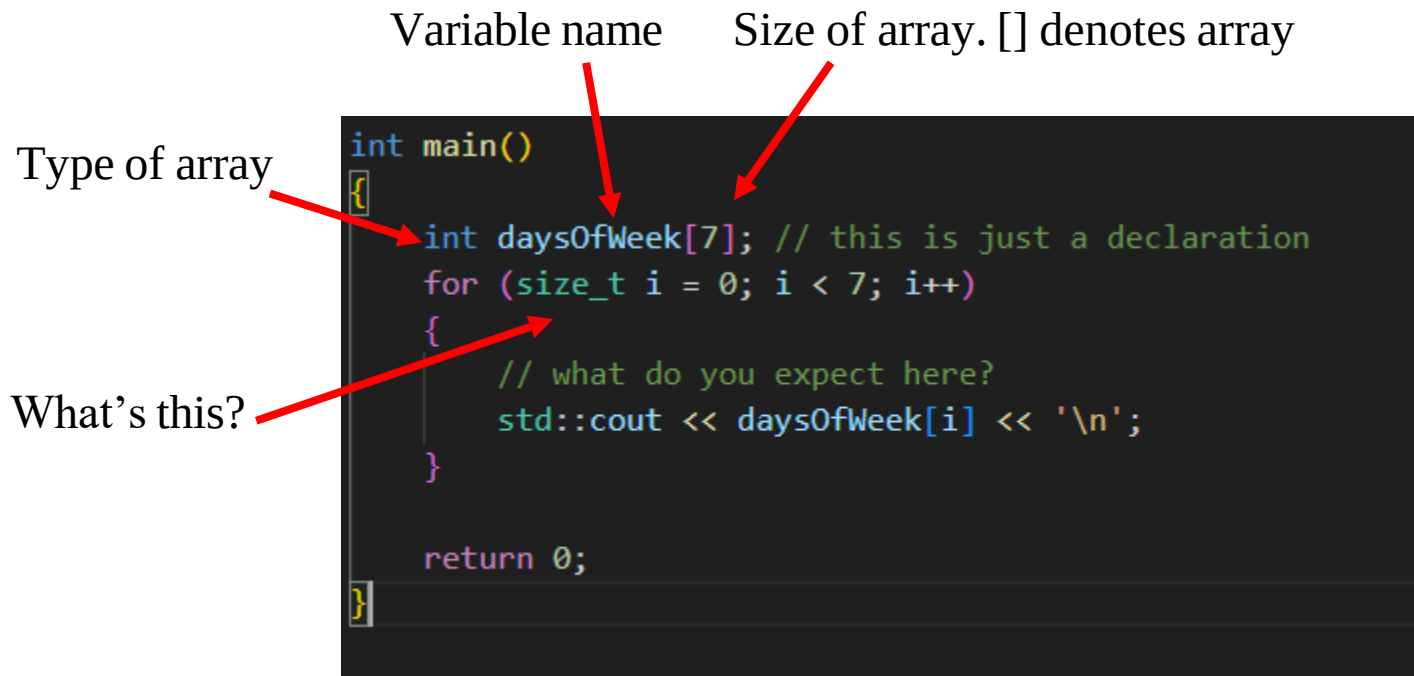
Variable name Size of array. [] denotes array

Type of array

What's this?

```
int main()
{
    int daysOfWeek[7]; // this is just a declaration
    for (size_t i = 0; i < 7; i++)
    {
        // what do you expect here?
        std::cout << daysOfWeek[i] << '\n';
    }

    return 0;
}
```



- What output do you see?

Array size



- Can be specified with constant variable (**const**)
const int size = 20;
- Constants cannot be changed
- Constants must be initialized when declared
- Also called named constants or read-only variables

Declaring multiple arrays of same type



Use comma separated list, like regular variables

```
int days[ 7 ], months[ 12 ];
```

Arrays

- Change the following in the previous code

```
int daysOfWeek[7];
```

to the following

```
int daysOfWeek[7] = {};
```

- What's the output?
- Now, if you change it to: `int daysOfWeek[7] = {0};` What's the output?
- Change it to: `int daysOfWeek[7] = {-1};` What's the output?
- Change to `int daysOfWeek[7] = {-1, 0, 1, 2, 3, 4, 5};`
- Finally to `int daysOfWeek[7] = {-1, 0, 1, 2, 3, 4, 5, 6};`

Arrays – What if you don't know
the size of the array
upfront?

We call array creation like this – “Array is created on the stack”

Arrays



- Make sure you don't access elements beyond the size of the array

```
int main()
{
    // this is now a definition
    int daysOfWeek[7] = {-1, 0, 1, 2, 3, 4, 5};

    for (size_t i = 0; i < 10; i++)
    {
        // what do you expect here?
        std::cout << daysOfWeek[i] << '\n';
    }

    return 0;
}
```

Passing array to functions – NOT RECOMMENDED



```
void printArray(bool classOnDay[7])
{
    std::cout << "=====\n";
    for (size_t day = 0; day < 7; day++)
    {
        std::cout << "Print: day=" << day << ", isClassOnDay=" << classOnDay[day] << '\n';
    }
}

int main()
{
    // this is now a definition
    bool classOnDay[7] = {true, false, false, false, false, true, true};
    printArray(classOnDay);

    return 0;
}
```

Passing array to functions

- Pass size separately to array
- What is the output?
- Similar to aliasing (not technically) in python. Will go into detail next

```
void foo(int arr[], const size_t n)
{
    for (size_t i = 0; i < n; i++)
    {
        arr[i] = i*2;
    }
}

int main(int argc, char ** argv)
{
    int j = 5;
    int arr[4] = {0};
    foo(arr, 4);

    for (size_t i = 0; i < 4; i++)
    {
        std::cout << arr[i] << '\n';
    }

    return 0;
}
```

Multidimensional array

- 2D array

```
int main()
{
    // this is now a definition
    double travelTime[7][5] = {};

    for (size_t day = 0; day < 7; day++)
    {
        for (size_t timeslot = 0; timeslot < 5; timeslot++)
        {
            double duration = 0;
            std::cin >> duration;
            travelTime[day][timeslot] = duration;
            std::cout << "day=" << day << ", timeslot="
                << timeslot << ", duration="
                << travelTime[day][timeslot] << '\n';
        }
    }

    return 0;
}
```

- 3D array: `double d3Arr[7][5][3] = {};`
- You can create n-D array by adding square brackets

Multidimensional array

Multiple subscripts

`a[ i ][ j ]`

- Tables with rows and columns
- Specify row, then column
- Also called “Array of arrays”

`a[0]` is an array of 4 elements

`a[0][0]` is the first element of that array

	Column 0	Column 1	Column 2	Column 3
Row 0	<code>a[0][0]</code>	<code>a[0][1]</code>	<code>a[0][2]</code>	<code>a[0][3]</code>
Row 1	<code>a[1][0]</code>	<code>a[1][1]</code>	<code>a[1][2]</code>	<code>a[1][3]</code>
Row 2	<code>a[2][0]</code>	<code>a[2][1]</code>	<code>a[2][2]</code>	<code>a[2][3]</code>

Diagram illustrating the structure of a 2D array (Table) with rows and columns. The array is named `a`. The first subscript (row) is `0`, and the second subscript (column) is `0`. The element at `a[0][0]` is the first element of the first row.

Labels and arrows pointing to the first element (`a[0][0]`):

- Array name (points to `a`)
- Row subscript (points to `0`)
- Column subscript (points to `0`)

Multidimensional array



Initializers grouped by row in braces

```
int b[ 2 ][ 2 ] = { { 1, 2 }, { 3, 4 } };
```

Row 0	1	2
Row 1	3	4

```
int b[ 2 ][ 2 ] = { { 1 }, { 3, 4 } };
```

1	0
3	4

Multidimensional array



- Referenced like normal

cout << b[0][1]; Outputs **0**

- Cannot reference using commas

cout << b[0, 1]; Syntax error

- Function prototypes

- Must specify sizes of subscripts

- First subscript not necessary, as with single-scripted arrays

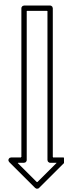
- **void printArray(int[][3]);**

1	0
3	4

Self Study : What is ENUM?

Enumerations – named ints

- Enums can be treated as named ints (old way).
 - We will consider this
- Modern and better way is to use 'enum class'
 - I will leave this as a reading for you
- Unless specified, the first item in enum list is assigned a value of 0 implicitly as shown below



```
enum DAYS {MONDAY = 0, TUESDAY, WEDNESDAY,  
           THURSDAY, FRIDAY, SATURDAY, SUNDAY};
```

```
int main()
{
    enum DAYS {MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY, SUNDAY};
    int i = -1;
    while(true) {
        int i = DAYS::MONDAY;
        cin >> i;
        if(i == -2) {
            break;
        }
        switch (i)
        {
            case MONDAY:
            case TUESDAY:
                cout << "class at 0830\n";
                break;
            case FRIDAY:
                cout << "class at 0840\n";
                break;
            case WEDNESDAY:
            case THURSDAY:
            case SATURDAY:
            case SUNDAY:
                cout << "No class\n";
                break;
            default:
                cout << "Invalid input\n";
                break;
        }
    }
}
```

Enumeration – named ints

- What should happen with the following code?

```
int main()
{
    enum DAYS {MONDAY = 1, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY = 10, SUNDAY};
    int i = -1;
    while(true) {
        int i = DAYS::MONDAY;
        cin >> i;
        if(i == -2) {
            break;
        }
        switch (i)
        {
            case MONDAY:
            case TUESDAY:
                cout << "class at 0830\n";
                break;
            case FRIDAY:
                cout << "class at 0840\n";
                break;
            case WEDNESDAY:
            case THURSDAY:
            case SATURDAY:
            case SUNDAY:
                cout << "No class\n";
                break;
            default:
                cout << "Invalid input\n";
                break;
        }
    }
}
```